

Semana 8: Comunicação e Fluxo de Dados entre Telas Mobile

O guia definitivo para passagem de parâmetros e retorno de dados usando o framework Flutter.

Mapa de Objetivos da Aula

1

Compreender o fluxo contínuo de dados (A Ida e A Volta).

2

Dominar a passagem de parâmetros via Construtores de classes.

3

Implementar a palavra-chave `await` em fluxos assíncronos.

4

Utilizar o comando `Navigator.pop` com variáveis de retorno.

O Problema: Por que as telas precisam conversar?

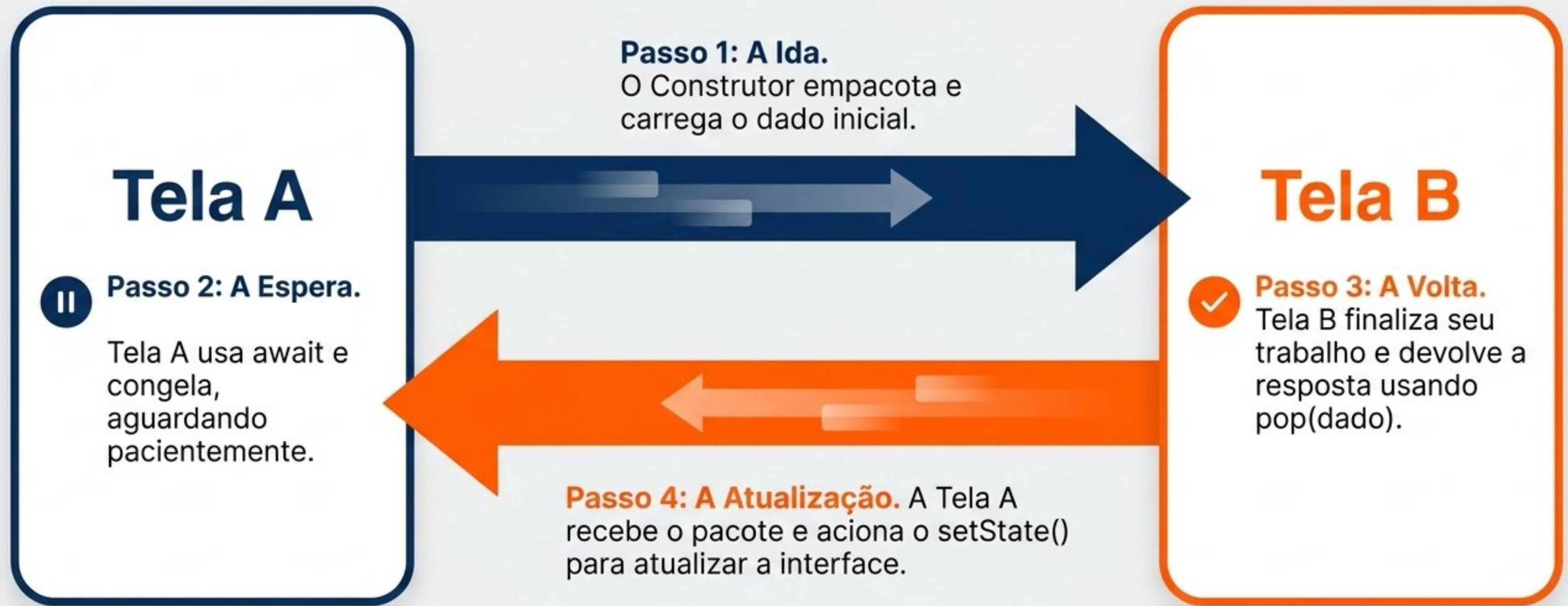
Na Semana 7, aprendemos a navegar entre telas. Porém, na vida real, a navegação **não é vazia; ela carrega contexto**. Telas isoladas não formam um aplicativo útil.



Matriz de Evolução Técnica

	Semana 7 (Navegação Cega)	Semana 8 (Navegação com Carga)
Ação Avançar	<code>push(novaTela())</code> (Abre sem enviar nada)	<code>push(novaTela(dado: objeto))</code> (Abre injetando um parâmetro)
Ação Voltar	<code>pop()</code> (Apenas destrói a tela atual)	<code>pop(resultado)</code> (Destrói a tela e envia um pacote para trás)
Comportamento	Síncrono (O código não aguarda a tela fechar)	Assíncrono (<code>await</code> congela a execução até o usuário retornar)

O Modelo Mental: A Jornada do Dado



Passo 1: A Ida (Enviando via Construtores)

Ao invocar a nova tela, injetamos a informação diretamente no momento em que seu Widget é criado.

A variável não sofrerá mutação direta aqui.

O Dart exige garantir que o dado nunca chegue nulo.

```
1 class TelaDetalhes extends StatelessWidget {  
2   final Tarefa tarefa;  
3  
4   const TelaDetalhes({required this.tarefa});  
5 }
```



Passo 2: A Espera (A Magia do `await`)

O código Dart é executado em milissegundos. Para garantir que a Tela Principal não continue rodando antes do usuário terminar de preencher o formulário, criamos uma sala de **espera**.



```
final resultado = await Navigator.push(...)
```


Passo 3: A Volta (Retornando Dados com pop)



1

O GPS interno — Diz ao Flutter exatamente onde estou na árvore do aplicativo para saber para onde voltar.

```
Navigator.pop(context, novoObjeto);
```

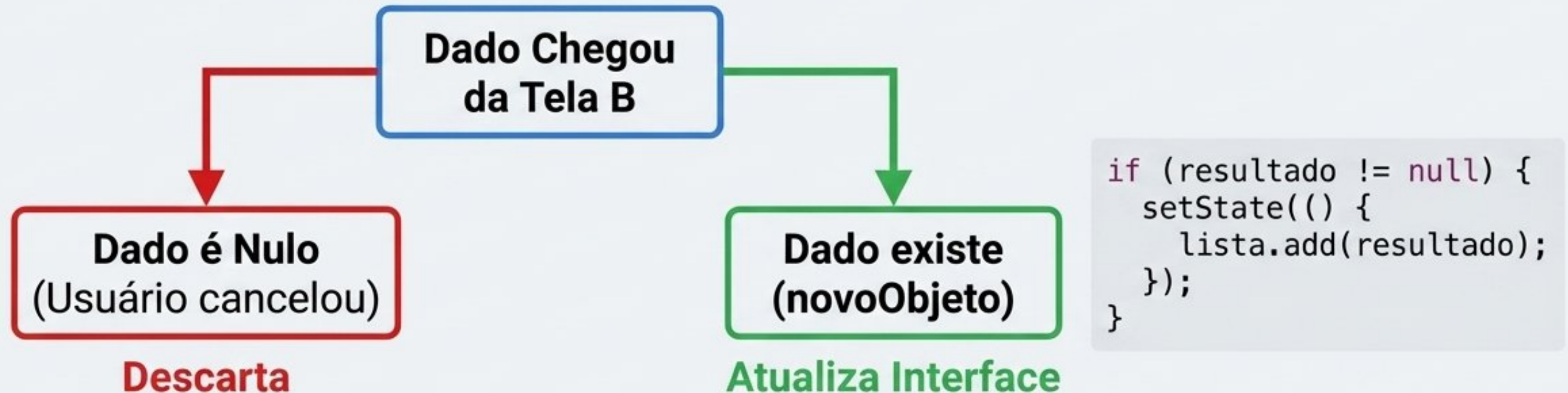
2

O Pacote — A variável contendo a informação (ex: uma nova Tarefa ou Produto) que vai viajar no túnel de retorno.

O comando de fechar a tela ganha uma bagagem. Em vez de voltar de mãos vazias, a Tela B devolve um objeto recém-criado ou editado.

Passo 4: Fechando o Ciclo (Atualizando a Tela A)

A execução do código volta imediatamente para a linha abaixo do **await**. Agora, precisamos processar a bagagem que chegou.



Regra de Ouro: Sempre verifique se o resultado não é nulo antes de atualizar a tela.

Síntese Prática: A Arquitetura do Mini-App

O Modelo



Classe **Produto**.
Define o que é o objeto
e quais atributos
viajam pelas telas.

A Listagem (Tela Principal)



Um **ListView** que
consome uma lista em
memória e usa **setState**
ao receber um retorno.

A Coleta (Formulário)



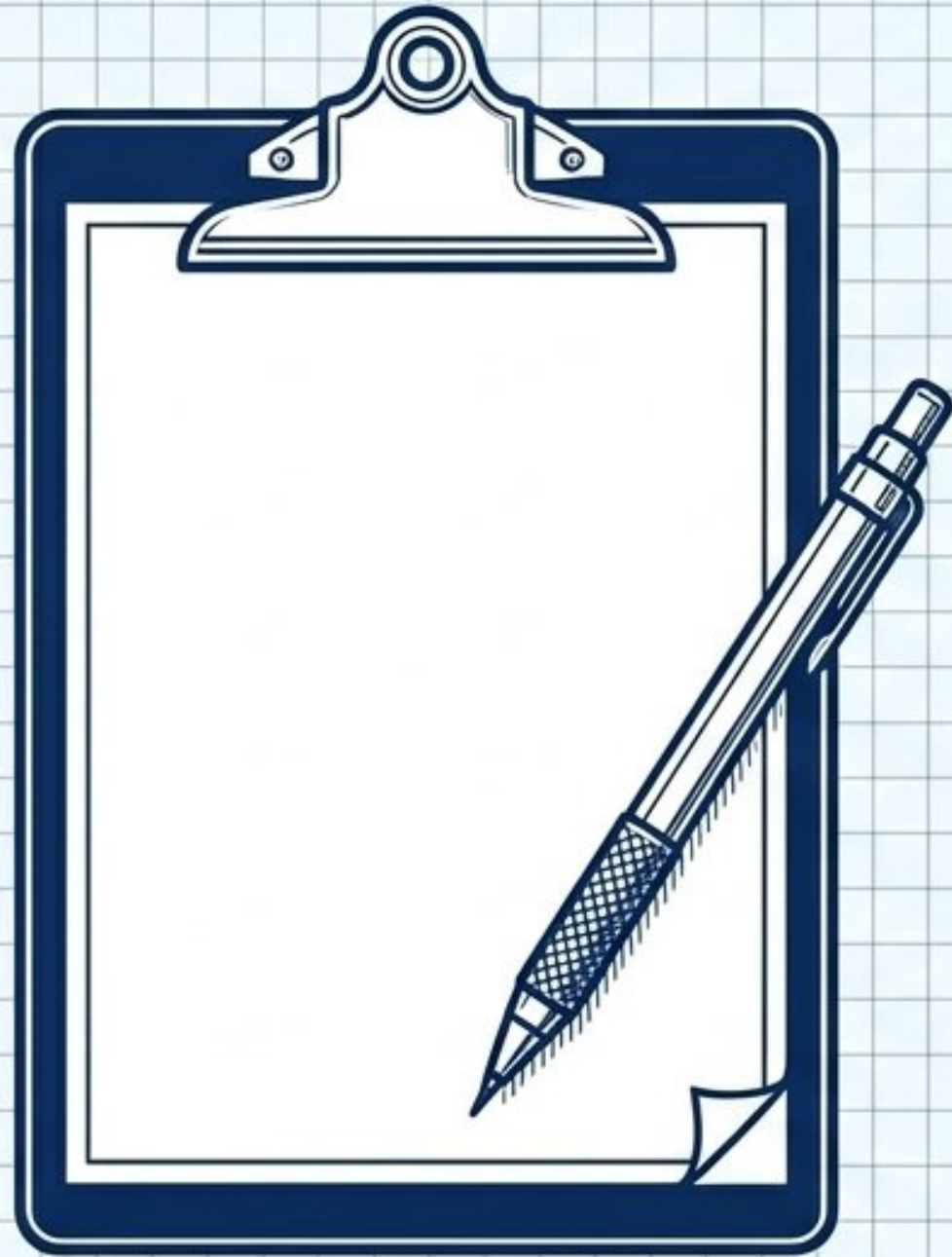
Múltiplos **TextFields**
que instanciam um
novo **Produto** e
enviam via **pop()**.

Atividade da Semana: Mini-App de Gestão

Missão: Criar um fluxo de aplicativo funcional conectando duas telas através da passagem e retorno de um objeto (Tarefas ou Produtos).

- [] **Requisito 1:** Tela A implementada contendo uma lista local em memória de itens.
- [] **Requisito 2:** Tela B contendo um formulário de criação (ao menos dois campos de texto).
- [] **Requisito 3:** Botão Salvar na Tela B encapsula os dados e os retorna utilizando `Navigator.pop()`.
- [] **Requisito 4:** Tela A utiliza `await` para receber os novos dados e atualiza a interface via `setState()`.

Atividade Verificadora: Desenhe a Arquitetura



Desafio Final: Antes de codificar, mapeie a lógica.

- Desenhe (em papel ou software) o diagrama arquitetural do fluxo que você programará.
- Trace visualmente o caminho do dado: Onde ele nasce? Como viaja para a Tela B? E como retorna para a lista da Tela A?
- Identifique graficamente os exatos pontos onde os comandos **push()**, **await** e **pop()** são disparados nesse ecossistema.